



Факультет компьютерных наук

Исследование и
предпринимательство в
искусственном интеллекте

Москва 2026

**Организация кооперативного восприятия сцены в группе автономных
агентов на основе обмена визуальной информацией
Cooperative Visual Perception in Multi-Agent Systems**

Студент: Вольхин Данил Федорович

Научный руководитель: Копылов Иван Станиславович

Предметная область и термины

КОНТЕКСТ РАБОТЫ

Работа находится на стыке multi-robot systems, fleet management, LLM-агентов и цифрового двойника.

AMR/AGV-флот

группа мобильных роботов, которым нужны общая карта, статусы и согласованные маршруты

VDA5050

промышленный формат обмена между fleet manager и роботами

MCP

стандартный tool-интерфейс между LLM-агентами и внешними сервисами

Digital twin

Isaac Sim и ROS 2 позволяют проверить маршрут до физического внедрения

Предмет работы не низкоуровневое управление колесами, а архитектура, которая переволит смысловую задачу человека в исполнимый контур миссий.

КЛЮЧЕВОЙ РАЗРЫВ

Одиночный робот видит локальную ситуацию, но решение часто должно менять поведение всей группы.

Существующие fleet management системы надежно диспетчеризуют миссии, но не дают высокоуровневого семантического планирования и интерфейса на естественном языке.

Локальное восприятие

камера, лидар и вычисления ограничены одним роботом

Разрозненные контуры

симуляция, VDA5050, LLM и UI обычно живут отдельно

Нет реактивной оркестрации

сложно быстро перестроить миссии роя по наблюдениям

Cooperative perception

алгоритмы дают обмен признаками и детекциями, но редко интегрируются с промышленным fleet management

VDA5050 и fleet management

дают надежную диспетчеризацию, но требуют формальных API-команд

LLM-агенты

умеют планировать и вызывать инструменты, но нуждаются в ограниченном исполнимом контуре

Вклад работы: воспроизводимая интеграция LLM-слоя, MCP-инструментов, VDA5050, ROS 2 и цифрового двойника Isaac Sim в одну исполнимую архитектуру.

Цель и задачи

ЦЕЛЬ РАБОТЫ

Создать AgentsSwarm: программный комплекс, который переводит запрос на естественном языке в скоординированные миссии группы роботов.

КАК ЦЕЛЬ ДЕКОМПОЗИРОВАНА

Архитектура

8 компонентов: интерфейс, Orchestrator, vLLM, MCP-слой и роботический контур

Агентное планирование

Router, Planner, PlanRunner и агенты для карт, роботов и маршрутов

Интеграционные протоколы

3 асинхронных MCP-сервера,
VDA5050-совместимые заказы и ROS 2

Проверка и оптимизация

контур в Isaac Sim, pytest/smoke-проверки и дистилляция SmoIVLA

Объект: многоагентная робототехническая система; методы: системный анализ, микросервисное проектирование, симуляционный эксперимент.

Требования и ограничения

ЧТО ДОЛЖНА ВЫДЕРЖИВАТЬ АРХИТЕКТУРА

Естественный язык

пользователь формулирует задачу без ручной сборки API-вызовов

VDA5050

исполнение оформляется как совместимый заказ миссии

MCP

LLM вызывает ограниченный каталог инструментов

ROS 2 + Isaac Sim

проверка идет в симуляционном роботическом контуре

Streaming

статусы, tool-вызовы и ответ видны пользователю

Воспроизводимость

компоненты разнесены как сервисы и фиксируются версиями

Ограничение принципиально: LLM не помещается в servo-loop робота; она работает на уровне плана и инструментов, а выполнение остаётся в роботическом контуре.

СТЕК ПРОГРАММНОГО ПРОЕКТА

Backend и агенты

Python, FastAPI, OpenAI Agents SDK, FastMCP, pytest

Frontend и данные

React, Chakra UI, Celery, Redis Streams, RabbitMQ, PostgreSQL, MinIO

LLM-инференс

vLLM Service, Qwen2.5-Instruct, OpenAI-compatible API, Data Parallel на 2 × Tesla V100

Роботический контур

NVIDIA Isaac Sim, ROS 2 Jazzy, Nav2, rosbridge, Mosquitto, VDA5050

Стек выбран под разделение ролей: веб-интерфейс, агентное планирование, LLM-инференс и роботическое исполнение связаны через REST, WebSocket, MCP и MQTT.

Инфраструктура развёртывания

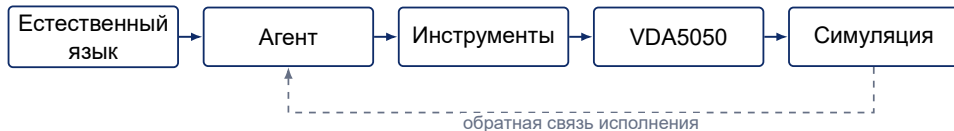
Таблица 1 — Технические характеристики вычислительной инфраструктуры

Назначение	Провайдер	Конфигурация	Сервисы
Симуляция	Selectel	4 vCPU, 16 ГБ RAM, RTX 4090 24 ГБ VRAM	Isaac Sim, ROS 2, Mission Control
Микросервисы	Selectel	4 vCPU, 8 ГБ RAM, 128 ГБ SSD	Interface, Orchestrator, Mission Dispatch, Redis, RabbitMQ, MinIO, PostgreSQL
LLM-кластер	MTS Cloud	2 узла × Tesla V100-PCIE 32 ГБ VRAM	vLLM Data Parallel, Qwen2.5-Instruct

Сетевые контуры: HTTP/REST между сервисами, WebSocket для стриминга, MQTT для VDA5050, gRPC для vLLM Data Parallel.

КЛЮЧЕВОЙ ПРИНЦИП

LLM не управляет роботом напрямую: LLM планирует поверх ограниченного набора инструментов.



Совместимость

MCP, VDA5050, ROS 2,
OpenAI-compatible API

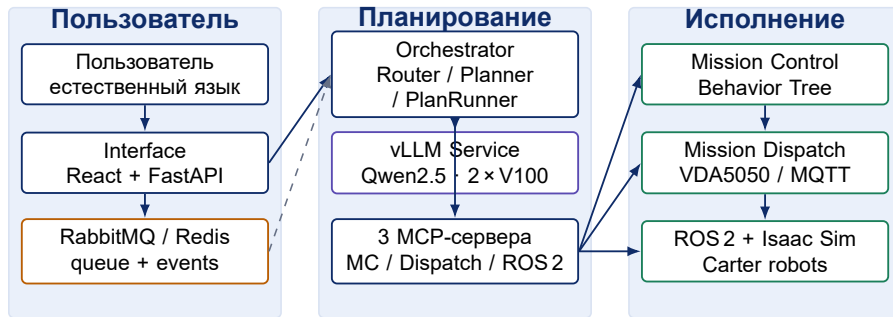
Наблюдаемость

шаги плана и tool-вызовы
попадают в поток событий

Воспроизводимость

компоненты изолированы как
сервисы

Общая архитектура AgentsSwarm



8

самостоятельных компонентов

UI, Orchestrator, vLLM, MCP, ROS 2, Isaac

Sim, Mission Control/Dispatch, SmoVLA

3

логических слоев

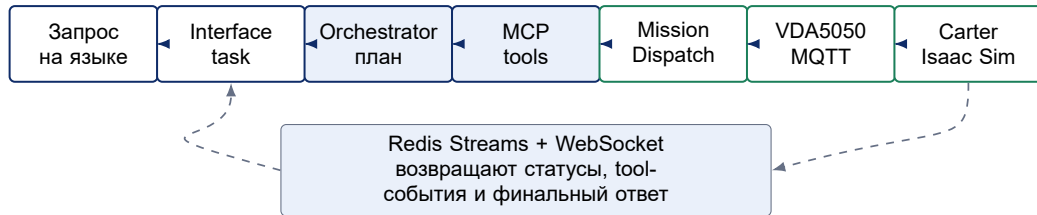
восприятие, планирование, исполнение

4

протокола связи

REST, WebSocket, MQTT, MCP

Сквозной маршрут команды

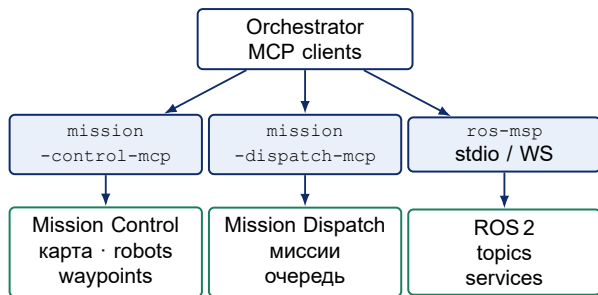


Пользовательский запрос превращается в план, затем в вызовы MCP-инструментов, затем в VDA5050 order через Mission Dispatch и MQTT-контур робота в Isaac Sim.

Orchestrator: агентная архитектура



MCP-слой: стандартный интерфейс к роботическому контуру



ЗОНЫ ОТВЕТСТВЕННОСТИ

- ▶ `mission-control-mcp`: карта, waypoint-граф, статусы.
- ▶ `mission-dispatch-mcp`: отправка и мониторинг миссий.
- ▶ `ros-msp`: ROS 2 topics через rosbridge WebSocket.

Почему это важно

Оркестратор не зависит от конкретной реализации роботического контура и работает через tool-интерфейс.

Роботический контур: Isaac Sim, ROS 2 и VDA5050

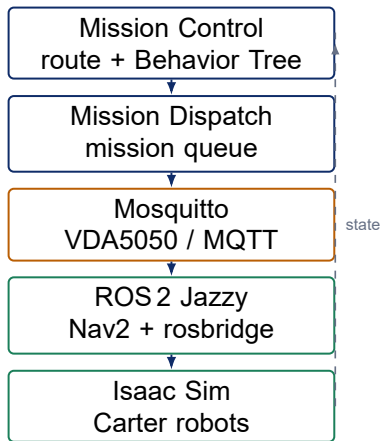


Рисунок 1 — Схема интеграции Isaac Sim, ROS 2 и VDA5050 в симуляционном контуре

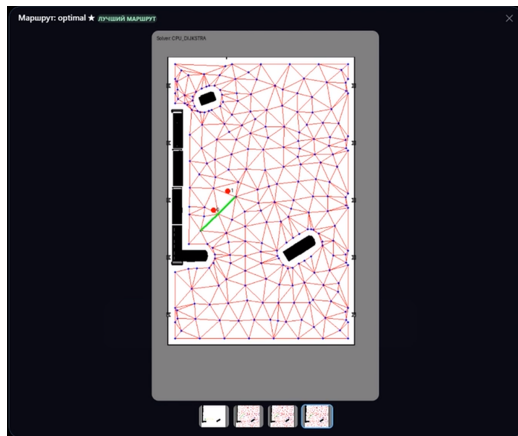
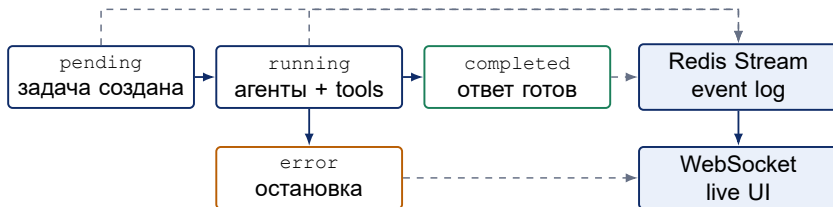


Рисунок 2 — Окупранси map и waypoint graph сцены Carter Warehouse

Потоковая модель событий



Жизненный цикл

pending → *running* → *completed* или *error*

Наблюдаемость

события `stream_chunk`, `routing_event`,
`tool_event`, `agent_reply`



Рисунок 3 — QR-код на видео-демо

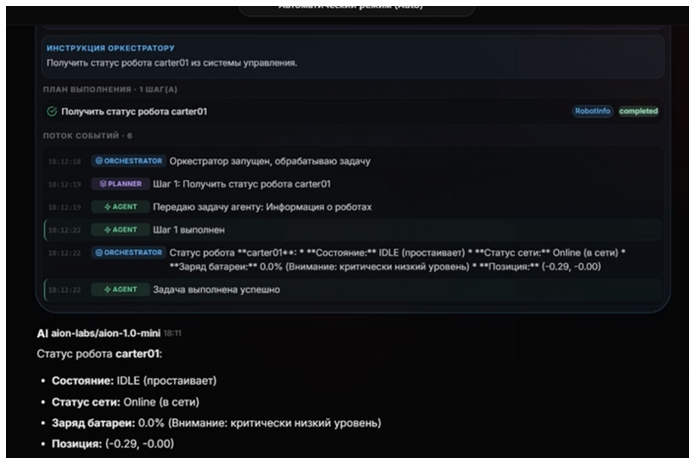


Рисунок 4 — Экран TracePanel: трассировка агентного маршрута

SmolVLA: перспектива — кастомный VDA5050 action-handler



Рисунок 5 — Навигация + бортовая автономность

Идея: VDA5050 order содержит поле `actions` в узлах маршрута; тип `vla_action` запускает локальный инференс SmolVLA вместо навигационной команды. Робот останавливается, выполняет задание, продолжает миссию.

Целевая платформа: NVIDIA Jetson Orin Nano (8 ГБ VRAM). Оптимизация SmolVLA делает бортовой инференс достижимым. Таблица 2 — Ключевые результаты оптимизации SmolVLA

Метрика	Teacher	Student	Изменение
Параметры	450M	≈1M	×443 сжатие
VRAM	6,0ГБ	<0,5ГБ	×12 снижение
Латентность	450 мс	18 мс	×25 ускорение
R^2	0,841	0,826	−1,8%

Результат: AgentsSwarm расширяется от «доставить в точку В» до «доставить и выполнить физическое задание» — от NL-запроса до бортовой манипуляции.

Примечание: эксперимент — `lerobot/libero`; перенос на Jetson Orin Nano требует отдельного профилирования.

Полученные результаты

Таблица 3 — Компоненты AgentsSwarm: срезы разработки и результаты

Модуль	Срез / объём	Результат
vLLM Service	v0.2.9, 32 коммита	Qwen2.5-Instruct развёрнут как OpenAI-compatible сервис на 2 × Tesla V100.
Ros2 контур	форки и настройка окружения	Mission Control/Dispatch, VDA5050/MQTT, ROS 2 Jazzy исполняют миссию, а Isaac Sim реализует миссию в физической симуляции.
MCP-серверы	3 async-сервера	Orchestrator получает tool-интерфейс к Mission Control, Mission Dispatch и ros-msp.
Orchestrator	v0.5.6, 61 коммит	Router классифицирует запрос, Planner строит план, PlanRunner исполняет его через агентов.
Interface	v0.5.0, 168 коммитов	Пользователь ставит задачу, запускает агентный сценарий и видит поток выполнения в веб-интерфейсе.
Тестирование	66 тестов	Проверены ключевые сценарии Orchestrator; Interface закрыт route, WebSocket и smoke-проверками.
SmoIVLA Tools	v0.2.3, 6 коммитов	Показаны сжатие параметров, снижение VRAM и ускорение при падении R^2 на 1.8%.

Итог: сквозной маршрут от естественного языка до исполнимой VDA5050-миссии собран, описан и проверен в симуляционном роботическом контуре.

Спасибо за внимание. Готов ответить на вопросы.

